What is a Stream?

A stream is a sequence of bytes flowing in or out of a program.

- Input stream → data flows *into* the program
- Output stream → data flows *out of* the program

Standard Streams in C++

C++ provides built-in streams through the <iostream> library:

Stream	Description
cin	Standard input (keyboard)
cout	Standard output (console)
cerr	Standard error (unbuffered)
clog	Standard log (buffered error output)

Example:

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cout << "Enter a number: ";
    cin >> x;
    cout << "You entered: " << x << endl;
    return 0;
}
```

Types of Streams

1. Input Stream (istream)

Used for reading data

Example: cin

2. Output Stream (ostream)

Used for writing data

Example: cout

3. File Streams (fstream)

Used for file handling:

Class	Purpose
ifstream	Read from file
ofstream	Write to file
fstream	Read & write

File Stream Example

```
#include <iostream>
#include <fstream>
using namespace std;
```

Athar Ahmed

```
int main() {
    ofstream outFile("D:\\TraingFramework\\C++\\Files\\example.txt");

    if (!outFile.is_open()) {
        cerr << "Error opening file!" << endl;
        return 1;
    }

    outFile << "Hello, file!" << endl;

    cout << "File written successfully." << endl;

    return 0;
}
```

Stream Operators

Operator	Use
>>	Extraction (input)
<<	Insertion (output)

Key Features of Streams

- Abstraction: You don't worry about device details
- Type safety: Automatically handles data types
- Extensible: You can create custom streams

Buffering

- cout is buffered (faster, but may delay output)
- cerr is unbuffered (immediate output)

C++, exception handling

In C++, exception handling is used to manage runtime errors in a controlled way using:

- try → block where code may cause an error
- throw → used to signal an error
- catch → handles the error

Basic Syntax

```
try {
    // risky code
    throw exception;   // throw error
}
catch(type variable) {
    // handle error
```

Athar Ahmed

```
}
```

Simple Example

```
#include <iostream>
using namespace std;

int main() {
    try {
        int age = 15;

        if (age < 18) {
            throw "Not eligible to vote";
        }

        cout << "Eligible to vote";
    }
    catch (const char* msg) {
        cout << "Error: " << msg;
    }

    return 0;
}
```

How It Works

1. Code inside try executes
2. If an error occurs → throw is triggered
3. Control jumps to matching catch block
4. Program continues after catch

Throwing Different Types

```
try {
    throw 10;   // integer exception
}
catch (int e) {
    cout << "Integer exception: " << e;
}
```

Multiple Catch Blocks

```
try {
    throw 3.14;
}
catch (int e) {
    cout << "Integer";
```

Athar Ahmed

```
}
catch (double d) {
    cout << "Double: " << d;
}
catch (...) {
    cout << "Unknown exception";
}
```

Example: Division by Zero

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 0;

    try {
        if (b == 0) {
            throw "Division by zero error";
        }
        cout << a / b;
    }
    catch (const char* msg) {
        cout << msg;
    }

    return 0;
}
```

Standard Exceptions

C++ provides built-in exceptions via <exception>:

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main() {
    try {
        throw runtime_error("Something went wrong");
    }
    catch (exception &e) {
        cout << e.what();
    }
}
```

Athar Ahmed

```
#include <iostream>
#include <fstream>
#include <stdexcept>
using namespace std;

int main() {
    int number;

    try {
        // ----------- INPUT STREAM (cin) -----------
        cout << "Enter a positive number: ";
        cin >> number;

        // Input validation using exception
        if (cin.fail()) {
            throw runtime_error("Invalid input (not a number)");
        }

        if (number < 0) {
            throw runtime_error("Negative number not allowed");
        }

        // ----------- OUTPUT FILE STREAM -----------
        ofstream outFile("data.txt");

        if (!outFile) {
            throw runtime_error("Unable to open file for writing");
        }

        outFile << "User entered: " << number << endl;
        outFile.close();

        cout << "Data written to file successfully.\n";

        // ----------- INPUT FILE STREAM -----------
        ifstream inFile("data.txt");

        if (!inFile) {
            throw runtime_error("Unable to open file for reading");
        }

        string line;
```

Athar Ahmed

```
        cout << "\nReading from file:\n";

        while (getline(inFile, line)) {
            cout << line << endl;
        }

        inFile.close();
    }
    catch (const exception &e) {
        // ----------- ERROR HANDLING -----------
        cerr << "Error: " << e.what() << endl;
    }

    cout << "\nProgram ended safely.";
    return 0;
}
```

Full combined Code

```
#include <iostream>
#include <fstream>
#include <stdexcept>
using namespace std;

int main() {
    int number;

    try {
        // ----------- INPUT STREAM (cin) -----------
        cout << "Enter a positive number: ";
        cin >> number;

        // Input validation using exception
        if (cin.fail()) {
            throw runtime_error("Invalid input (not a number)");
        }

        if (number < 0) {
            throw runtime_error("Negative number not allowed");
        }

        // ----------- OUTPUT FILE STREAM -----------
        ofstream outFile("data.txt");
```

Athar Ahmed

```
        if (!outFile) {
            throw runtime_error("Unable to open file for writing");
        }

        outFile << "User entered: " << number << endl;
        outFile.close();

        cout << "Data written to file successfully.\n";

        // ----------- INPUT FILE STREAM -----------
        ifstream inFile("data.txt");

        if (!inFile) {
            throw runtime_error("Unable to open file for reading");
        }

        string line;
        cout << "\nReading from file:\n";

        while (getline(inFile, line)) {
            cout << line << endl;
        }

        inFile.close();
    }
    catch (const exception &e) {
        // ----------- ERROR HANDLING -----------
        cerr << "Error: " << e.what() << endl;
    }

    cout << "\nProgram ended safely.";
    return 0;
}
```

What This Program Demonstrates

1. Streams
   - cin → takes input from user
   - cout → displays output
   - ofstream → writes to file
   - ifstream → reads from file

---

2. Exception Handling

Ammar Ahmed

- throw runtime_error(...) → generates error
- try → wraps risky code
- catch (exception &e) → handles all standard exceptions
- e.what() → prints error message

Example Runs
Valid Input
Enter a positive number: 25
Data written to file successfully.

Reading from file:
User entered: 25

Program ended safely.

Invalid Input
Enter a positive number: abc
Error: Invalid input (not a number)

Program ended safely.

Negative Input
Enter a positive number: -5
Error: Negative number not allowed

Program ended safely.

Athar Ahmed